

Universidad Nacional Autónoma de México
Facultad de Ciencias

Laboratorio de Compiladores
Manual de Laboratorio

Práctica 1: De regex a NFA

Semestre: 2026-2

Índice

1. Objetivos	1
2. Implementación	1
2.1. Estructura de la solución	1
2.2. Validación de la solución	3
3. Entregables	5
4. Evaluación	6
4.1. Reporte	6
4.2. Video de validación	7
4.3. Puntos extra	8
5. Consideraciones finales	9

1. Objetivos

El objetivo de esta práctica es que el estudiante:

- Comprenda y aplique los algoritmos de construcción (como el algoritmo de Thompson) para transformar una expresión regular básica en un autómata finito no determinista (NFA).
- Implemente estructuras de datos eficientes para representar estados y transiciones (incluyendo transiciones ϵ).
- Desarrolle un motor de simulación que utilice el NFA resultante para validar si una cadena de texto pertenece al lenguaje definido por la expresión regular.

2. Implementación

2.1. Estructura de la solución

En esta práctica, se espera que los estudiantes implementen un sistema que permita transformar una expresión regular en un autómata finito no determinista (NFA) y luego simular dicho NFA para validar cadenas de texto. La estructura de la solución debe cubrir los siguientes requerimientos:

- **Parser de Expresiones Regulares:** Se debe poder leer una expresión regular infija de concatenación implícita y convertirla en una representación interna adecuada para su procesamiento. Esto incluye la conversión de concatenación implícita a explícita y la conversión a notación postfija utilizando el algoritmo de Shunting-Yard.
- **Constructor de NFA:** Implementar el algoritmo de Thompson para construir un NFA a partir de la representación interna de la expresión regular.
- **Simulador de NFA:** Simulación del NFA resultante para determinar si una cadena de texto es aceptada o no por la expresión regular.
- **Contrato de entrada/salida:** El programa recibirá un parámetro (-r o -t) para indicar la acción a realizar (conversión de regex a posfijo con concatenación explícita o validación de cadenas de texto), seguido de la expresión regular y/o la(s) cadenas de texto a validar, respectivamente. El programa debe imprimir el resultado de la conversión o validación en la salida estándar.

Para la conversión de regex a posfijo, la salida debe ser la expresión regular en notación postfija con concatenación explícita. Para la validación de cadenas de texto, la salida debe ser “1” si la cadena es aceptada por el NFA y “0” si no lo es.

Por ejemplo, para la expresión regular $a(b|c)^*$ y la cadena de texto `abcb`, el programa debería imprimir “1” indicando que la cadena es aceptada por el NFA construido a partir de la expresión regular. En cambio, para la cadena de texto `ac`, el programa debería imprimir “0” indicando que la cadena no es aceptada por el NFA.

Un ejemplo de ejecución para la conversión de regex a posfijo sería:

- Windows:

```
@(
  "(ab)*"
  "ab"
  "aba"
  "abab"
) -join "`n" | .\regex_to_nfa.exe -t
```

- Linux:

```
printf '%s\n' "(ab)*" "ab" "aba" "abab" | ./regex_to_nfa -t
```

La salida esperada para ambos casos sería 101.

Para el caso de la conversión de regex a posfijo, la salida esperada para la expresión regular `(ab)*` sería `ab*`. El comando de ejecución sería:

```
echo "(ab)*" | .\regex_to_nfa -r
```

Para facilitar la implementación, se proporciona un ejemplo del código principal (`main.c`) que muestra cómo se espera que se estructure el programa y cómo se deben manejar los argumentos de entrada:

```
#include "regex.h"
#include "nfa.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <getopt.h>

void print_postfix(regex r)
{
    for (int i = 0; i < r.size; i++)
    {
        printf("%c", r.items[i].value);
    }
    printf("\n");
}

void test_strings_stdin(const char *regex_str)
{
    regex r = parse_regex(regex_str);
    nfa n = regex_to_nfa(r);

    char buf[1024];
    while (fgets(buf, sizeof(buf), stdin))
    {
        buf[strcspn(buf, "\r\n")] = '\0';
        int result = match_nfa(n, buf, strlen(buf));
    }
}
```

```

        printf("%d", result ? 1 : 0);
    }
    printf("\n");
}

int main(int argc, char *argv[])
{
    int opt;
    char regex_str[1024];

    while ((opt = getopt(argc, argv, "rt")) != -1)
    {
        switch (opt)
        {
            case 'r':
                if (!fgets(regex_str, sizeof(regex_str), stdin))
                    return 1;
                regex_str[strcspn(regex_str, "\r\n")] = '\0';
                print_postfix(parse_regex(regex_str));
                return 0;
            case 't':
                if (!fgets(regex_str, sizeof(regex_str), stdin))
                    return 1;
                regex_str[strcspn(regex_str, "\r\n")] = '\0';
                test_strings_stdin(regex_str);
                return 0;
            default:
                fprintf(stderr, "Usage: %s -r | -t\n", argv[0]);
                return 1;
        }
    }

    fprintf(stderr, "Usage: %s -r | -t\n", argv[0]);
    return 1;
}

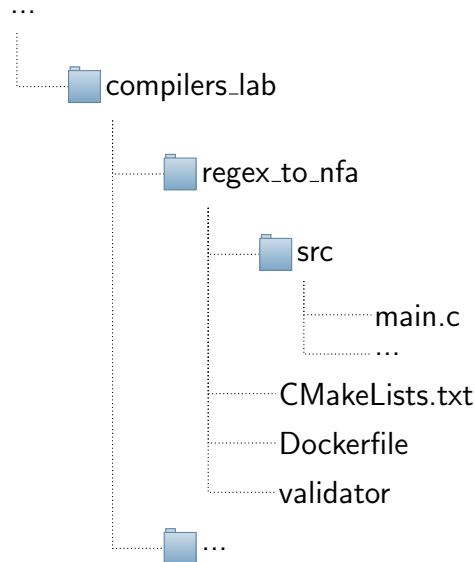
```

2.2. Validación de la solución

Para la validación de la solución, se utilizará un sistema de pruebas que comprobará la salida del programa con diferentes casos de prueba. Para esto, se compilará el programa utilizando **CMake**, **Make** y **musl** para asegurar la portabilidad y consistencia de los resultados. El sistema de pruebas se ejecutará dentro de un contenedor Docker, lo que garantiza un entorno controlado y reproducible para la evaluación. El contenedor se construirá a partir de un **Dockerfile** que incluye todas las dependencias necesarias para compilar y ejecutar el programa, así como el sistema de pruebas. Al ejecutar el contenedor, se compilará el código del alumno y se ejecutarán las pruebas automáticamente, proporcionando retroalimentación

inmediata sobre la corrección de la solución.

Es importante que el programa del alumno cumpla con los requisitos de entrada y salida especificados, ya que el sistema de pruebas se basará en estos formatos para validar la solución. Para que el proceso de validación sea exitoso, se debe tener la siguiente estructura de archivos en el proyecto:



En caso de no estar familiarizado con CMake o Make, se recomienda revisar la documentación oficial de cada herramienta para entender cómo funcionan. A continuación, se proporcionará un ejemplo básico de cómo se podría configurar CMakeLists.txt para compilar un programa en C con varios archivos fuente.

```
cmake_minimum_required(VERSION 3.16)
project(regex_to_nfa C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)

add_executable(regex_to_nfa
    ./src/main.c
    ./src/nfa.c
    ./src/regex.c
)
```

Ejemplo de CMakeLists.txt

El archivo Dockerfile se proporcionará a continuación, el cual se encargará de configurar el entorno necesario para compilar y ejecutar el programa, así como para ejecutar las pruebas de validación. Para poder ejecutar el contenedor, se requiere tener Docker instalado en el sistema. En Windows, se puede instalar Docker Desktop (requiere WSL) y en Linux, se puede instalar Docker Engine siguiendo las instrucciones específicas para cada distribución.

```
# Use the latest Alpine image as the base
```

```
FROM alpine:latest

# Install build tools (gcc, musl-dev, make, cmake) for compiling the
# code
RUN apk add --no-cache gcc musl-dev make cmake

# Create a directory for the application
WORKDIR /app

# Copy the source code into the container
COPY . .

# Container starts by compiling the code and running the validator
CMD ["sh", "-c", "cmake . && make && ./validator"]
```

Dockerfile

Para construir la imagen de Docker, se debe ejecutar el siguiente comando en la terminal, asegurándose de estar en el directorio donde se encuentra el Dockerfile:

```
docker build -t regex_to_nfa_validator .
```

Comando para construir la imagen de Docker

Una vez construida la imagen, se puede ejecutar el contenedor con el comando **docker run**, lo que iniciará el proceso de validación automáticamente. Se recomienda ejecutar el contenedor con la opción **-rm** para que se elimine automáticamente después de su ejecución, manteniendo así el sistema limpio de contenedores innecesarios.

```
docker run --rm regex_to_nfa_validator
```

Comando para ejecutar el contenedor de Docker

3. Entregables

Para esta práctica, se debe subir al repositorio de GitHub el código fuente completo del proyecto, incluyendo todos los archivos fuente, el archivo CMakeLists.txt y cualquier otro recurso necesario para compilar y ejecutar el programa. Asegúrese de que la estructura del proyecto sea clara y siga las convenciones establecidas.

Además, es importante incluir en la entrega de Classroom el reporte de la práctica en formato PDF. El reporte debe seguir la siguiente estructura:

- Portada con los datos del estudiante y la práctica.
- Validación. Aquí irá una fotografía de la actividad realizada en el laboratorio o en su caso un enlace al video para la opción B de validación.
- Descripción de la implementación. Incluya detalles sobre las estructuras de datos utilizadas, los algoritmos implementados y cualquier decisión de diseño relevante.

- Dificultades encontradas durante la implementación y cómo se resolvieron.
- Resultados obtenidos. Aquí se debe incluir la salida del validador (la captura de pantalla de la terminal con la ejecución del contenedor Docker).

4. Evaluación

4.1. Reporte

Se evaluará el reporte siguiendo la siguiente rúbrica:

Criterio (total)	Excelente (100 % - 81 %)	Bueno (80 % - 51 %)	Insuficiente (50 % - 1 %)	No válido (0 %)
Portada (1pt)	Completa y correcta	Completa pero con errores menores	Incompleta	Ausente
Validación (1pt)	Fotografía o video incluido	-	-	No se agregó ni fotografía ni video
Implementación (4pts)	Descripción clara y detallada	Descripción adecuada pero con falta de detalles	Descripción incompleta o confusa	Ausente
Dificultades (2pts)	Descripción clara y soluciones detalladas	Descripción adecuada pero con falta de detalles	Descripción incompleta o confusa	Ausente
Resultados (2pts)	Captura de pantalla con salida exitosa del validador	-	-	Ausente

Se otorgará la calificación total según la evaluación de cada uno de los criterios mencionados. **Para que la práctica sea evaluada, deberá contener de forma obligatoria la fotografía o el video de la validación, así como la captura de pantalla con la salida del validador.** La falta de cualquiera de estos elementos resultará en una calificación de 0 para la práctica, independientemente de los demás criterios evaluados.

De forma aleatoria, se seleccionarán algunos reportes para una revisión de código más detallada, donde se ejecutará el validador con el código fuente entregado para verificar que la solución implementada cumple con los requisitos establecidos en la práctica. **En caso de encontrar discrepancias entre el reporte y el código entregado, se anulará la calificación de la práctica.** Se considera una falta grave el no entregar un reporte que refleje fielmente el trabajo realizado. Es fundamental que el reporte sea un documento honesto y preciso que describa el proceso de implementación y validación de la solución, ya que esto es parte integral del aprendizaje y la evaluación en esta práctica.

4.2. Video de validación

En caso de optar por la opción B de validación, se debe agregar un enlace al video en el reporte, asegurándose de que el video sea accesible. Los puntos a evaluar en el video de validación serán los siguientes:

Criterio (total)	Excelente (100 % - 81 %)	Bueno (80 % - 51 %)	Insuficiente (50 % - 0 %)
Audio y video (1pt)	Claros y sin interrupciones	Claros pero con interrupciones menores	Difíciles de entender o con interrupciones significativas
Explicación teórica (4pts)	Explicación clara y completa de los objetivos y la implementación	Explicación adecuada pero con falta de detalles o claridad en algunos puntos	Explicación confusa o incompleta
Explicación técnica (4pts)	Descripción detallada de la implementación, incluyendo estructuras de datos y algoritmos utilizados	Descripción adecuada pero con falta de detalles o claridad en algunos puntos	Descripción confusa o incompleta
Resultados (1pt)	Muestra claramente la salida del validador y explica los resultados obtenidos	Muestra la salida del validador pero con explicación limitada de los resultados	No muestra la salida del validador o no explica los resultados

Para esta práctica, la explicación teórica debe incluir los siguientes puntos:

- Explicación de un ejemplo de expresión regular, mostrando paso a paso cómo se transforma en un NFA. Esto debe incluir los algoritmos de conversión de concatenación implícita a explícita, conversión a postfijo (algoritmo de Shunting-Yard), construcción de NFA (algoritmo de Thompson) y simulación del NFA.
- Explicación de la simulación del NFA, detallando cómo se manejan las transiciones ϵ y cómo se determina si una cadena de texto es aceptada por el NFA.

En cuanto a la explicación técnica, se espera que el video cubra los siguientes aspectos:

- Implementación de los algoritmos de concatenación implícita a explícita, conversión a postfijo, construcción de NFA y simulación del NFA.
- Flujo de trabajo de la implementación, desde la lectura de la expresión regular hasta la validación de las cadenas de texto, destacando cómo se integran los diferentes componentes del sistema.
- Compilación y ejecución del programa usando el contenedor Docker, incluyendo instrucciones para ejecutar las pruebas.

4.3. Puntos extra

No todo es negativo, también se pueden obtener puntos extra por aspectos adicionales que se consideren relevantes para la práctica. En este laboratorio, se otorgarán puntos extra por la implementación de características adicionales que no fueron requeridas pero que demuestren un esfuerzo adicional y una comprensión más profunda del tema.

Para considerar la asignación de puntos extra, agregue al reporte una sección titulada **Puntos Extra**, donde se mencionen (no es necesario que se describan a detalle) las características adicionales y/o puntos relevantes que se hayan implementado. A continuación, se detallan los criterios para la asignación de puntos extra:

- Generales:

- (+3 pts) Código limpio y bien documentado, con comentarios claros y una estructura de proyecto organizada.
- (+2 pts) Implementación de pruebas unitarias adicionales para validar cada componente del programa de manera independiente.
- (+1 pt) Reporte de calidad excepcional, con una redacción clara, sin errores ortográficos y con un formato profesional.
- (+1 pt) Redacción del reporte en un idioma adicional al español, como inglés, demostrando habilidades de comunicación en otro idioma. (En caso de ser redactado en inglés, se podrá omitir el reporte en español).
- (+1 pt) Inclusión de un análisis de complejidad temporal y espacial de la solución implementada, demostrando una comprensión profunda de los algoritmos utilizados.

- Específicos:

- (+3.5 pts) Implementación de una interfaz gráfica para visualizar el NFA resultante.
- (+1.5 pts) Soporte para expresiones regulares extendidas, como conjuntos de caracteres o cuantificadores.
- (+1 pt) Optimización del algoritmo de simulación del NFA para mejorar su rendimiento.
- (+3 pts) Resolución a mano de una regex propuesta mostrando el proceso de construcción del NFA paso a paso, así como la simulación de una cadena válida y una cadena no válida. La longitud de la regex propuesta debe ser de al menos 15 caracteres (sin contar paréntesis ni concatenaciones), y debe incluir todos los operadores vistos.
- (+3 pts) Implementación de un sistema de carga/guardado de NFAs en formato JSON, permitiendo a los usuarios guardar su trabajo y cargarlo posteriormente para continuar con su desarrollo o validación.

- Video de validación:

- (+3 pts) Exposición del video en idioma inglés, demostrando habilidades de comunicación en otro idioma. (Si se opta por esta opción, se podrá omitir la explicación en español, pero se recomienda incluir subtítulos en español para asegurar la comprensión de todos los evaluadores).
- (+2 pts) Edición de video de calidad y creatividad excepcional. Esto incluye una presentación visual atractiva, uso efectivo de gráficos o animaciones para explicar conceptos, y una narrativa clara y envolvente que mantenga el interés del espectador a lo largo de todo el video.
- (+1.5 pts) Inclusión de una sección de preguntas y respuestas al final del video, donde se aborden posibles dudas o preguntas comunes relacionadas con la práctica, demostrando una comprensión profunda del tema y la capacidad de anticipar las necesidades de los espectadores.

Nota: Los puntos extra no son acumulativos, es decir, solo serán validos para la práctica actual y no se podrán transferir a futuras prácticas o evaluaciones de teoría. Además, para quienes opten por la opción B de validación, los puntos extra (a excepción de los relacionados con el video) no se aplicarán para aumentar la calificación del video. Por ejemplo, si un estudiante obtiene 8.5 puntos en el video y .5 puntos extra en el reporte, su calificación final seguirá siendo 8.5 puntos.

5. Consideraciones finales

Esta es nuestra primera práctica del semestre y sé que enfrentarse a conceptos abstractos, herramientas nuevas y un entorno de desarrollo distinto puede parecer abrumador. ¡No tengan miedo de experimentar! El verdadero aprendizaje ocurre cuando nos permitimos cometer errores y corregirlos.

Más allá de un resultado perfecto, lo que más nos interesa es que disfruten el camino y se lleven una buena experiencia. Si se sienten atascados, no se queden con la duda: apóyense en sus compañeros o escribanme un correo; ninguna pregunta es pequeña cuando se trata de aprender. Confío plenamente en su capacidad para superar este reto. ¡Mucho ánimo y éxito en este primer laboratorio!